COMPLETE END-TO-END ENGINEERING, OTP/SMTP & SECURITY SPECIFICATION

Offline AI Platform — Full Technical Documentation

Exhaustive architectural reference covering the Initial Auth Build Specification (`AUTH_BUILD_SPEC.md`), Reference Repo Inheritance (`offline_ai` & `offline_frontend`), Redis OTP Engine (`otp_service.py`), Gmail STARTTLS SMTP Service (`mailer_service.py`), Phase 1.5 Security Hardening, Phase 2 Complete 21-Table PostgreSQL Schema, All REST API Endpoints & CSV Bulk Pipelines, Super Admin Security Audit (#1–#14), Global ITU-T E.164 Phone & Student Login Support, and Docker Containerization & DB Migration.

BACKEND REPOSITORY
D:\Work\TechnoExponent\offline_ai_backend

FRONTEND REPOSITORY
D:\Work\TechnoExponent\offline_frontend

READ-ONLY REFERENCE REPO
D:\Work\TechnoExponent\offline_ai

RUNTIME & DEPLOYMENT
FastAPI + Postgres 16 + Redis 7 + Docker

21	90+	7	18	60+	4
POSTGRESQL TABLES	MOUNTED ROUTE PATHS	RBAC USER ROLES	SECURITY & AUDIT FIXES	E.164 COUNTRY CODES	DOCKER CONTAINERS

Table of Contents

1. **Section 1:** Origin & Architecture Inherited from `AUTH_BUILD_SPEC.md`
2. **Section 2:** OTP Engine (`otp_service.py`) & Gmail SMTP Service (`mailer_service.py`)
3. **Section 3:** Authentication Flows, Crypto (`security.py`) & Frontend Contract
4. **Section 4:** Phase 1.5 — Auth Security & Reliability Hardening
5. **Section 5:** Phase 2 — Complete 21-Table Database Schema (`offlineai.*`)
6. **Section 6:** Complete REST API Reference (All 6 Routers & Bulk Upload Pipelines)
7. **Section 7:** Super Admin Security Audit & Precautions (Items #1–#14)
8. **Section 8:** Global International Phone Numbers (ITU-T E.164) & Student Login
9. **Section 9:** Docker Containerization & Local-to-Docker DB Migration
10. **Section 10:** Complete File Inventory & Operational Runbook

1. Origin & Architecture Inherited from AUTH_BUILD_SPEC.md

1.1 Relationship Between the Three Repositories

As documented in Section 1 & Section 5 of `D:\Work\TechnoExponent\offline_ai_backend\AUTH_BUILD_SPEC.md`, the ecosystem consists of three distinct codebases with strict boundaries:

- **1. Read-Only Reference Microservice (`D:\Work\TechnoExponent\offline_ai`):**
  - This repository is a **downstream content/lesson-generation microservice**. It contains *zero* login, registration, OTP, or email-sending code.
  - Instead, it only *reads* users and academic entities via read-only SQLAlchemy mirror models defined in `offline_ai/app/models/offlineai_core.py` (such as `OfflineUser` , `OfflineClass` , and `offline_ai/app/models/school.py` 's `sections` and `school_class_section`) and verifies JWTs issued elsewhere via `get_current_offline_user` in `offline_ai/app/core/security.py` .
  - Crucially, `OfflineUser` in `offlineai_core.py` specifies `__tablename__ = "users"` , `__table_args__ = {"schema": "offlineai"}` , and `id = Column(UUID(as_uuid=True), primary_key=True)` with uppercase Postgres enum status (`ACTIVE` , `INACTIVE` , `DELETED`).
  - **Architectural Decision Inherited:** Rather than creating tables in Postgres's `public` schema or using string UUIDs (`String(36)`), `offline_ai_backend` creates all tables inside the **`offlineai`** PostgreSQL schema using native PostgreSQL

`UUID(as_uuid=True)` primary keys and `offlineai.user_status` enum. This guarantees that when both backends point to the same PostgreSQL database, `offline_ai`'s read-only mirror models resolve against live data with zero modifications.
    - We also ported the model mixin patterns (`Base`, `UUIDMixin`, `TimestampMixin`, `SoftDeleteMixin`) from `offline_ai/app/models/base.py` into `offline_ai_backend/app/models/base.py`.
- **2. Pre-Built React Frontend (`D:\Work\TechnoExponent\offline_frontend`):**
    - A complete React 18 + Vite + MUI + Redux + `react-hook-form` + `yup` SPA wired to a fixed API contract in `src/utils/constants.js` (`API_URL`), `src/services/api/auth.api.js`, `src/services/actions/auth.action.js`, and `src/services/reducers/auth.reducer.js`.
    - The backend was built to conform to the frontend's exact request/response envelope expectations, and the frontend was subsequently enhanced only where explicitly requested (post-reset redirect and global E.164 phone number support).
- **3. Core Platform & Auth Backend (`D:\Work\TechnoExponent\offline_ai_backend`):**
    - Owns authentication, 2FA OTP generation & Gmail SMTP delivery, JWT issuance, password recovery, and complete CRUD across all 21 tables in `offlineai`.

1.2 Strict 5-Layer Separation of Concerns ("Different Microservices in Different Files")

Per Section 7.5 of `AUTH_BUILD_SPEC.md` ("*the python backend should have the different microservices in different files, its not that everything is ingested at once*"), the backend enforces strict module isolation where each concern lives in its own dedicated file:

Layer / Module	Responsibility	Strict Prohibitions (What Must NOT Live Here)
HTTP Router Layer `app/api/v1/*.py`	Thin FastAPI route handlers: parses Pydantic request bodies, calls `app/services/*_service.py`, and wraps output in `success_response()`.	**NO** business logic, **NO** direct SQLAlchemy `db.execute()` queries, **NO** direct Redis calls, **NO** direct SMTP calls.
Auth Business Orchestrator `app/services/auth_service.py`	Orchestrates login, OTP verification, resend, forgot/reset password, and student login by calling `user_service`, `otp_service`, `mailer_service`, and `core.security`.	**NO** FastAPI `Request`/`Response` objects, **NO** inline Redis commands, **NO** raw `aiosmtplib` calls.
OTP Engine Service `app/services/otp_service.py`	Cryptographic 6-digit OTP generation, purpose-scoped Redis storage/TTL, brute-force attempt counters, 30s resend cooldowns, login session markers, and `revoke_otp()`.	Knows **NOTHING** about email, SMTP, users, SQLAlchemy, or HTTP.
Generic SMTP Mailer `app/services/mailer_service.py`	Exposes a single reusable function `send_email(*, to, subject, body)` over Gmail STARTTLS (`aiosmtplib`) with automatic `SMTP_DEV_MODE` console fallback.	Knows **NOTHING** about OTPs, users, Redis, or email templates (subject/body templates live in `auth_service.py`).
Data Access Services `app/services/user_service.py`, `school_service.py`, etc.	Encapsulates all raw SQLAlchemy 2.0 async queries against `offlineai.*` tables.	**NO** HTTP status codes or token encoding logic where pure DB access is expected.

2. OTP Engine (otp_service.py) & Gmail SMTP Service (mailer_service.py)

2.1 How Gmail SMTP Works & Why Google Apps Script Was Rejected

In Section 7 of `AUTH_BUILD_SPEC.md`, the design addressed whether OTP emails should be sent via a Google Apps Script webhook or direct SMTP:

- **No Google Apps Script:** Instead of external script webhooks, the backend connects directly to Gmail's official SMTP submission server (`smtp.gmail.com` on port `587` using `STARTTLS`) via Python's asynchronous `aiosmtplib` library and standard library `email.message.EmailMessage`.
- **Gmail 2FA + 16-Character App Password Requirement:** Google blocks standard account passwords on SMTP connections. To send real emails through Gmail SMTP, the sender's Google account must have 2-Step Verification enabled and generate a **16-character App Password** at `https://myaccount.google.com/apppasswords`.
- **Dual-Mode Execution (`SMTP_DEV_MODE` vs Live Gmail SMTP):**
  - Configured in `app/core/config.py`:
    - `SMTP_HOST: str = "smtp.gmail.com"`
    - `SMTP_PORT: int = 587` (STARTTLS encryption)
    - `SMTP_USERNAME: str` (Gmail address)
    - `SMTP_PASSWORD: str` (16-character Google App Password)
    - `SMTP_FROM_EMAIL: str` & `SMTP_FROM_NAME: str = "Auth Backend"`
    - `SMTP_DEV_MODE: bool = True`
  - When `SMTP_DEV_MODE=True` or `SMTP_PASSWORD` is empty, `mailer_service.py` prints a formatted banner with the recipient, subject, and 6-digit OTP directly to the server/Docker logs so the entire 2FA login and password reset flows can be tested offline without waiting on external SMTP credentials.
  - When `SMTP_DEV_MODE=False` and a valid 16-char `SMTP_PASSWORD` is provided in `.env`, `mailer_service.py` dispatches the email live via `aiosmtplib.send( ... , start_tls=True)`.

2.2 Complete Implementation of `app/services/mailer_service.py`

```
import aiosmtplib
from email.message import EmailMessage
from app.core.config import settings

async def send_email(*, to: str, subject: str, body: str) → None:
    if settings.SMTP_DEV_MODE or not settings.SMTP_PASSWORD:
        print(
            "\n"
            "═══════════ [DEV MODE] EMAIL NOT ACTUALLY SENT ═══════════\n"
            f"To:      {to}\n"
            f"Subject: {subject}\n"
            f"Body:\n{body}\n"
            "═══════════════════════════════════════════════════════════\n"
        )
        return

    message = EmailMessage()
    message["From"] = f"{settings.SMTP_FROM_NAME} <{settings.SMTP_FROM_EMAIL}>"
    message["To"] = to
    message["Subject"] = subject
    message.set_content(body)

    await aiosmtplib.send(
        message,
        hostname=settings.SMTP_HOST,
        port=settings.SMTP_PORT,
        username=settings.SMTP_USERNAME,
        password=settings.SMTP_PASSWORD,
        start_tls=True,
    )
```

2.3 Email Template Generator & Background SMTP Dispatch (`app/services/auth_service.py`)

As specified in Section 7.5 of `AUTH_BUILD_SPEC.md`, email content generation lives in `auth_service.py` so `mailer_service.py` remains a pure transport layer:

```
def _otp_email_body(code: str, *, purpose_label: str) → str:
    return (
        f"Your one-time verification code is: {code}\n\n"
        f"This code is for {purpose_label} and expires in a few minutes.\n"
        f"If you did not request this, you can safely ignore this email."
    )

async def _send_otp_email_in_background(
    *, purpose: OtpPurpose, to: str, subject: str, body: str
) → None:
    """Sends an OTP email off the request path (used by forgot-password and
    student login so response latency doesn't reveal whether the account
    exists) and rolls back the Redis OTP + cooldown keys if SMTP fails."""
    try:
        await send_email(to=to, subject=subject, body=body)
    except Exception as exc:
        print(f"[SMTP ERROR] Failed to send {purpose.value} OTP to {to}: {exc!r}")
        await revoke_otp(purpose, to)
```

2.4 Deep-Dive into the Redis OTP Engine (`app/services/otp_service.py`)

The OTP service manages the entire lifecycle of one-time passwords in Redis using purpose-isolated key namespaces:

- **Cryptographic Code Generation (`generate_otp_code`):** Uses Python's CSPRNG `secrets.choice("0123456789")` (never `random`) to generate a 6-digit string (`settings.OTP_LENGTH = 6`) matching the frontend's `/^\d{6}$/` validation regex.
- **Purpose Isolation (`OtpPurpose` Enum):**
  - `OtpPurpose.LOGIN = "login"`
  - `OtpPurpose.FORGOT_PASSWORD = "forgot_password"`
  - `OtpPurpose.STUDENT_LOGIN = "student_login"`

  Because every Redis key includes `{purpose.value}`, an OTP requested via Forgot Password (which requires no password) can **never** be replayed at `POST /api/v1/auth/verify` to log in.
- **Brute-Force Protection (`OTP_MAX_ATTEMPTS = 5`):** Every call to `verify_otp()` increments `otp_attempts:{purpose}:{email}` in Redis. If a user or attacker submits 5 wrong guesses within the 5-minute window, `verify_otp()` immediately deletes the OTP key, the attempts counter, and the login session key, forcing the user to re-authenticate from Step 1.
- **Constant-Time Verification:** Uses `secrets.compare_digest(stored_code, submitted_code)` to prevent byte-by-byte timing side-channel attacks on OTP verification.

Complete Redis Key Specification

Redis Key Pattern	Value	TTL	Lifecycle & Security Role
`otp:{purpose}:{identifier}`	6-digit string (e.g. `"482910"`)	`300s` (5 min) `OTP_EXPIRE_SECONDS`	Stores active OTP for `login` , `forgot_password` , or `student_login` . Deleted immediately upon successful verification, 5 failed attempts, or SMTP rollback.
`otp_attempts:{purpose}:{identifier}`	Integer counter (`1..5`)	`300s` (5 min)	Incremented via Redis `INCR` on every verification attempt. Invalidates the OTP if `attempts ≥ 5` (`OTP_MAX_ATTEMPTS`).
`otp_cooldown:{purpose}:{identifier}`	`"1"`	`30s` `OTP_RESEND_COOLDOWN_SECONDS`	Blocks OTP spam for 30 seconds (matching the frontend's 25-second resend countdown in `OtpVerification.jsx`).
`otp_login_session:{email}`	`"1"`	`300s` (5 min)	Created by `start_login_session()` only after Step 1 password check passes. Required by both `resend_login_otp()` and `verify_otp(LOGIN)` .

Redis Key Pattern	Value	TTL	Lifecycle & Security Role
`used_reset_token:{jti}`	`"1"`	`900s` (15 min) `JWT_RESET_TOKEN_EXPIRE_MINUTES`	Marks a password-reset JWT's UUID4 `jti` claim as consumed in `reset_password()`, guaranteeing single-use enforcement.

3. Authentication Flows, Crypto (security.py) & Frontend Contract

3.1 Cryptographic Design in `app/core/security.py`

- **Direct `bcrypt` Package (No `passlib`):** As mandated in Section 8 of `AUTH_BUILD_SPEC.md`, `passlib` is unmaintained and suffers from compatibility bugs with `bcrypt ≥ 4.0`. We use the native `bcrypt` library directly with `BCRYPT_ROUNDS = 12` and explicit `MAX_BCRYPT_PASSWORD_BYTES = 72` guards.
- **Anti-Enumeration Dummy Hash (`DUMMY_PASSWORD_HASH`):** At module load time, `security.py` computes a real cost-12 bcrypt hash of `b"not_a_real_password"`. In `auth_service.login()`, if `user_service.get_user_by_email()` returns `None`, the server still executes `verify_password(password, DUMMY_PASSWORD_HASH)` so an attacker cannot distinguish existing vs. non-existing emails via response timing.
- **Purpose-Scoped JWT Tokens (`TokenPurpose` Enum):** Every JWT signed by `security.py` carries an explicit `"purpose"` claim verified by `decode_token(token, expected_purpose= ... )`:
  - `TokenPurpose.ACCESS = "access"` — TTL `1440` minutes (24h). Carries `sub` (user UUID) and `user_type`.
  - `TokenPurpose.PASSWORD_RESET = "password_reset"` — TTL `15` minutes. Carries `sub` (user UUID) and a unique `jti` (UUID4) tracked in Redis `used_reset_token:{jti}`.
  - `TokenPurpose.GUARDIAN_SELECT = "guardian_select"` — TTL `15` minutes. Carries `sub="{country_code}:{phone}"`, `country_code`, and `phone` for multi-child student login selection.

3.2 Frontend Contract Alignment (Section 4 of `AUTH_BUILD_SPEC.md`)

Every response from the backend conforms to the exact JSON envelope expected by the frontend's Redux actions (`src/services/actions/auth.action.js`) and reducers (`src/services/reducers/auth.reducer.js`):

```
// app/core/response.py — Standard Envelope
{
  "success": true,
  "message": "Login successful.",
  "data": { ... },
  "verification_required": false,
  "errors": null
}
```

- **Custom Exception Interception (`register_exception_handlers` in `app/core/response.py`):** FastAPI's default `{"detail": "..."}` error body breaks the frontend's `error.response.data.errors || error.response.data.message` error reader. Therefore, `response.py` intercepts `AppError`, `StarletteHTTPException`, `RequestValidationError` (converting Pydantic field errors into a clean `{field: msg}` map under `errors`), SQLAlchemy `IntegrityError`, and catch-all `Exception`, wrapping every 4xx/5xx response in the exact same envelope.
- **How `/login` and `/verify` Exchange Context:**
  - The frontend's login form submits `{identifier, password, remember_me}` to `POST /api/v1/auth/login`.
  - `/login` verifies the password, starts the Redis login session, sends the OTP via `mailer_service`, and returns `{"success": true, "message": "OTP sent to your email.", "verification_required": true, "data": null, "errors": null}`.
  - The frontend stores the original login payload in Redux `verificationPayload` and, when the user enters the 6-digit OTP on `OtpVerification.jsx`, spreads `{ ... verificationPayload, otp}` into `POST /api/v1/auth/verify`.
  - `VerifyOtpRequest` in `app/schemas/auth.py` accepts `identifier` (or `email` fallback via `@model_validator`) and `otp`, ignoring extra spread fields (`password`, `remember_me`) via `extra="ignore"`.
  - Upon OTP verification, `_build_user_public_data()` returns the flat user payload expected by `auth.reducer.js`'s `getPayloadUser()`: `access_token`, `id`, `first_name`, `last_name`, `full_name`, `email`, `country_code`, `phone_number`, `phone`, `gender`, `user_type`, `role` (mirrored alongside `user_type`), `status`, `is_first_login`, `schools`, `school`, `assigned_schools`, `school_assignments`, and `permissions`.

4. Phase 1.5 — Auth Security & Reliability Hardening

Fix #1: Preventing 2FA-to-1FA Password Bypass on `/resend-otp` & `/verify` CRITICAL SECURITY FIX

Vulnerability: If `POST /api/v1/auth/resend-otp` only checked whether an email existed in PostgreSQL before calling `issue_otp(OtpPurpose.LOGIN, email)`, an attacker who did not know a user's password could skip `/login`, call `/resend-otp` to mint a valid login OTP, and complete login using only the 6-digit code.

Implementation in `otp_service.py` & `auth_service.py`:

- `auth_service.login()` calls `await start_login_session(email)` (setting Redis `otp_login_session:{email} = "1"` with 300s TTL) **only after** `verify_password()` succeeds.
- `auth_service.resend_login_otp()` checks `if not await has_login_session(email): return` — silently no-oping if Step 1 password authentication was not completed, preventing both 2FA bypass and account enumeration.
- `otp_service.verify_otp()` also enforces `if purpose == OtpPurpose.LOGIN and not await has_login_session(email): return False`, and deletes `otp_login_session:{email}` immediately when the OTP is consumed or max attempts are exceeded.

Fix #2: Preventing Unhandled 500 Crashes on >72-Byte Passwords HIGH RELIABILITY FIX

Vulnerability: C-level `bcrypt.hashpw()` and `bcrypt.checkpw()` raise `ValueError: password cannot be longer than 72 bytes` when given UTF-8 strings exceeding 72 bytes.

Implementation: Added `MAX_BCRYPT_PASSWORD_BYTES = 72` checks in `app/core/security.py` (`verify_password` returns `False` instead of raising; `hash_password` raises a clean error) and added Pydantic UTF-8 byte-length validators in `app/schemas/auth.py` and `app/schemas/users.py` (`PasswordCheckMixin`), returning a clean `422 Unprocessable Entity` envelope.

Fix #3: SMTP Failure Rollback (`revoke_otp`) & Background Dispatch RELIABILITY FIX

Vulnerability: Writing `otp_cooldown:{purpose}:{email}` before calling `send_email()` meant that if Gmail SMTP failed, the user received no email but was locked out by a `429 Too Many Requests` cooldown for 30 seconds.

Implementation: Added `revoke_otp(purpose, email)` in `otp_service.py` to delete the OTP, attempt counter, and cooldown keys on any `send_email()` exception in `login()`, `resend_login_otp()`, and `_send_otp_email_in_background()`.

Fix #4: Post-Password-Reset Redirect to Login (`ForgotPasswordPage.jsx` / `ResetPasswordPage.jsx`) FRONTEND FIX

Ensured that once `POST /api/v1/auth/reset-password` succeeds, the frontend clears temporary reset state and redirects the user cleanly back to the login page (`/`).

4.1 Five-Domain Security & Compliance Matrix

Security Domain	Threat / Audit Requirement	Code-Level Enforcement & Verification
1. Authentication Security	Plain-text password checking & OTP exposure in API responses	• **Zero Plain-Text Password Checks:** Passwords are hashed and verified exclusively via native `bcrypt` (`BCRYPT_ROUNDS = 12`) in `app/core/security.py`, with `DUMMY_PASSWORD_HASH` comparison on unknown emails to prevent timing attacks. • **Zero OTP Exposure in API Responses:** `/login`, `/resend-otp`, `/forgot-password`, and `/student/request-otp` never include the OTP code in the JSON response body (returning only `verification_required: true` or a status message). OTPs are delivered out-of-band via Gmail SMTP (or server console in `SMTP_DEV_MODE`).

Security Domain	Threat / Audit Requirement	Code-Level Enforcement & Verification
2. OTP Security Controls	Missing expiry, rate limiting, and single-use validation controls	• **Strict Expiry:** Every OTP in Redis expires automatically after `OTP_EXPIRE_SECONDS = 300` (5 minutes). • **Dual Rate Limiting & Brute-Force Lockout:** `otp_cooldown:{purpose}:{id}` blocks resends for `30s`, while `otp_attempts:{purpose}:{id}` tracks failed guesses and wipes the OTP + session after `OTP_MAX_ATTEMPTS = 5` wrong attempts. • **Single-Use Enforcement:** `verify_otp()` immediately deletes the OTP, attempt counter, cooldown, and login session keys upon match.
3. Password Reset & Complexity	Password reuse protection and strong password enforcement	• **Password Reuse Protection:** Both `reset_password()` and `change_password()` in `app/services/auth_service.py` check `verify_password(new_password, user.hashed_password)` and reject reuse of the current password with `400 Bad Request`. • **Strong Password Complexity:** `_validate_strong_password()` in `app/schemas/auth.py` and `PasswordCheckMixin` in `app/schemas/users.py` enforce 8–72 UTF-8 bytes, ≥1 uppercase (`[A-Z]`), ≥1 lowercase (`[a-z]`), ≥1 digit (`\d`), and ≥1 special character (`[^A-Za-z0-9]`). • **Single-Use Reset JWT:** Consumed reset token `jti` claims are recorded in Redis `used_reset_token:{jti}` (15-min TTL).
4. Access Control (RBAC)	User, school, academic, and location management APIs authorization restrictions	• **Live DB Status Verification:** `get_current_user` in `app/api/v1/auth.py` queries PostgreSQL on every request to verify `not user.is_deleted` and `user.status == 'ACTIVE'`. • **Admin Role Enforcement:** `get_current_admin_user_id` restricts all User Management & Bulk Upload endpoints (`users.py`), School mutations & exports (`schools.py`), Location mutations (`locations.py`), Academic mutations (`academics.py`), and Dashboard/Permission APIs (`dashboard.py`) strictly to administrative roles, blocking students/teachers/operators with `403 Forbidden`.
5. Data Protection (CSV & Stats)	CSV export endpoints and statistics APIs access controls & input validation	• **Admin-Only Access:** `GET /api/v1/school/export`, `GET /api/v1/dashboard/summary`, `GET /api/v1/reports/summary`, and all CSV bulk-upload endpoints require `Depends(get_current_admin_user_id)`. • **Strict Query Input Validation:** `search` is bounded to `max_length=100`, `status` is regex-validated (`^(?i)(active|inactive|deleted)$`), and location filters (`state_id`, `district_id`, `ulb_id`, `ward_id`) are strictly typed as `uuid.UUID`. • **CSV Formula Injection Defense (CWE-1236):** `_sanitize_csv_cell()` in `app/services/school_service.py` prefixes cells starting with `=`, `+`, `-`, `@`, `\t`, or `\r` with `'` to prevent spreadsheet formula execution.

5. Phase 2 — Complete 21-Table Database Schema (offlineai.*)

In Phase 2, `D:\Work\TechnoExponent\offline_ai_backend\sql\init.sql` and the SQLAlchemy models in `app/models/` were expanded to **21 tables** inside the `offlineai` schema. Every soft-deletable ORM model inherits a dual Python/SQL `@hybrid_property is_deleted` (checking `deleted_at is not None` in Python and `cls.deleted_at.is_not(None)` in SQL).

#	Table Name	ORM Model Class	Columns, Constraints & Relationships
1	`offlineai.users`	`User` (`models/user.py`)	`id` (UUID PK), `email` (UNIQUE NOT NULL), `hashed_password`, `first_name`, `last_name`, `username` (UNIQUE), `country_code` (DEFAULT '+91'), `phone_number`, `phone` (UNIQUE), `gender`, `user_type`, `status` (ENUM `ACTIVE`, `INACTIVE`, `DELETED`), `is_first_login`, `last_login_at`, `profile_picture_url`, `created_by` (FK users), `created_at`, `updated_at`, `deleted_at`, `deleted_by`.
2	`offlineai.states`	`State` (`models/location.py`)	`id` (UUID PK), `name` (VARCHAR 150), `code` (VARCHAR 50 UNIQUE), `status`, `created_at`, `updated_at`, `deleted_at`, `deleted_by`.
3	`offlineai.districts`	`District` (`models/location.py`)	`id` (UUID PK), `state_id` (FK `states.id` ON DELETE CASCADE), `name`, `code`, `status`, timestamps, soft-delete. `UNIQUE (state_id, code)`.
4	`offlineai.ulbs`	`Ulb` (`models/location.py`)	`id` (UUID PK), `district_id` (FK `districts.id` ON DELETE CASCADE), `name`, `code`, `status`, timestamps, soft-delete. `UNIQUE (district_id, code)`.
5	`offlineai.wards`	`Ward` (`models/location.py`)	`id` (UUID PK), `ulb_id` (FK `ulbs.id` ON DELETE CASCADE), `name`, `code`, `status`, timestamps, soft-delete. `UNIQUE (ulb_id, code)`.
6	`offlineai.classes`	`SchoolClass` (`models/academic.py`)	`id` (UUID PK), `name` (e.g. 'Class 6'), `code` ('CLS-6'), `has_streams` (BOOLEAN DEFAULT FALSE), `status`, timestamps, soft-delete.
7	`offlineai.sections`	`Section` (`models/academic.py`)	`id` (UUID PK), `class_id` (FK `classes.id` ON DELETE CASCADE), `name` ('Section A'), `code` ('SEC-A'), `status`, timestamps, soft-delete.
8	`offlineai.streams`	`Stream` (`models/academic.py`)	`id` (UUID PK), `class_id` (FK `classes.id` ON DELETE SET NULL), `name`, `code`, `status`, timestamps, soft-delete.
9	`offlineai.qualifications`	`Qualification` (`models/academic.py`)	`id` (UUID PK), `name` (VARCHAR 150 UNIQUE, e.g. 'B.Ed', 'M.Sc'), `code`, `status`, timestamps, soft-delete.
10	`offlineai.schools`	`School` (`models/school.py`)	`id` (UUID PK), `name`, `udise_code` (VARCHAR 100 UNIQUE), `country_code` (DEFAULT '+91'), `phone`, `state_id` (FK RESTRICT), `district_id` (FK RESTRICT), `ulb_id` (FK SET NULL), `ward_id` (FK SET NULL), `status`, timestamps, soft-delete.

#	Table Name	ORM Model Class	Columns, Constraints & Relationships
11	`offlineai.school_class_sections`	`SchoolClassSection` (`models/school.py`)	`id` (UUID PK), `school_id` (FK CASCADE), `class_id` (FK CASCADE), `section_id` (FK CASCADE), `custom_name`, `custom_code`, `status`, timestamps, soft-delete. `UNIQUE (school_id, class_id, section_id)`.
12	`offlineai.subjects`	`Subject` (`models/academic.py`)	`id` (UUID PK), `name`, `code`, `class_id` (FK `classes.id` CASCADE), `school_id` (nullable FK `schools.id` CASCADE), `status`, timestamps, soft-delete.
13	`offlineai.subject_sections`	`SubjectSection` (`models/academic.py`)	`id` (UUID PK), `subject_id` (FK `subjects.id` CASCADE), `section_id` (FK `sections.id` CASCADE). `UNIQUE (subject_id, section_id)`.
14	`offlineai.chapters`	`Chapter` (`models/academic.py`)	`id` (UUID PK), `subject_id` (FK `subjects.id` CASCADE), `name`, `code`, `status`, timestamps, soft-delete.
15	`offlineai.topics`	`Topic` (`models/academic.py`)	`id` (UUID PK), `chapter_id` (FK `chapters.id` CASCADE), `name`, `code`, `status`, timestamps, soft-delete.
16	`offlineai.user_schools`	`UserSchool` (`models/assignments.py`)	`id` (UUID PK), `user_id` (FK `users.id` CASCADE), `school_id` (FK `schools.id` CASCADE), `role` (DEFAULT 'school_admin'), timestamps, soft-delete. `UNIQUE (user_id, school_id)`.
17	`offlineai.teacher_qualifications`	`TeacherQualification` (`models/assignments.py`)	`id` (UUID PK), `teacher_id` (FK `users.id` CASCADE), `qualification_id` (FK `qualifications.id` CASCADE). `UNIQUE (teacher_id, qualification_id)`.
18	`offlineai.teacher_assignments`	`TeacherAssignment` (`models/assignments.py`)	`id` (UUID PK), `teacher_id` (FK CASCADE), `school_id` (FK CASCADE), `class_id` (FK CASCADE), `subject_id` (FK CASCADE), timestamps, soft-delete. `UNIQUE (teacher_id, school_id, class_id, subject_id)`.
19	`offlineai.classroom_operator_assignments`	`ClassroomOperatorAssignment` (`models/assignments.py`)	`id` (UUID PK), `operator_id` (FK CASCADE), `school_id` (FK CASCADE), `class_id` (FK CASCADE), `section_id` (FK CASCADE), timestamps, soft-delete. `UNIQUE (operator_id, school_id, class_id, section_id)`.
20	`offlineai.student_profiles`	`StudentProfile` (`models/assignments.py`)	`id` (UUID PK), `user_id` (UNIQUE FK `users.id` CASCADE), `school_id` (FK CASCADE), `class_id` (FK RESTRICT), `section_id` (FK RESTRICT), `roll_number`, `academic_year`, `gender`, `dob`, `guardian_first_name`, `guardian_last_name`, `guardian_email`, `guardian_country_code` (DEFAULT '+91'), `guardian_phone`, timestamps, soft-delete.

#	Table Name	ORM Model Class	Columns, Constraints & Relationships
21	`offlineai.role_permissions`	`RolePermission` (`models/assignments.py`)	`id` (UUID PK), `role_name` (UNIQUE: `SUPER_ADMIN`, `SCHOOL_ADMIN`, `CONTENT_ADMIN`, `TEACHER`, `CLASSROOM_OPERATOR`, `STUDENT`), `label`, `readonly` (BOOLEAN), `permission_codes` (JSONB array), `created_at`, `updated_at`.

6. Complete REST API Reference (All 6 Routers & Bulk Upload Pipelines)

All 6 router modules are mounted under `/api/v1` in `app/main.py`. Many endpoints expose dual route aliases (e.g., both singular `/api/v1/school` and plural `/api/v1/schools`) so every page in `offline_frontend` resolves seamlessly.

6.1 Auth Router (`app/api/v1/auth.py` — Prefix `/api/v1/auth`)

Method	Exact Route Path	Auth Guard	Request Body & Response Contract
POST	`/api/v1/auth/login`	Public	Body: `{identifier, password, remember_me?}`. Verifies Bcrypt password (with dummy-hash timing protection), starts Redis login session, issues 6-digit OTP, sends email via `mailer_service`, returns `verification_required: true`.
POST	`/api/v1/auth/verify`	Public + Session	Body: `{identifier, otp, ...context}`. Verifies `otp:login:{email}` + `otp_login_session:{email}`, updates `last_login_at`, returns 24h `access_token` + enriched user object.
POST	`/api/v1/auth/resend-otp`	Public + Session	Body: `{email, phone?, role?, flow?}`. Requires active `otp_login_session:{email}`; enforces 30s cooldown; issues & emails fresh login OTP.
POST	`/api/v1/auth/forgot-password`	Public	Body: `{email}`. Anti-enumeration safe: always returns 200 OK; if active user exists, issues `otp:forgot_password:{email}` and sends email in background task.
POST	`/api/v1/auth/forgot-password/verify`	Public	Body: `{email, otp}`. Verifies & consumes forgot-password OTP; returns 15-min single-use JWT `{reset_token}` with unique `jti`.
POST	`/api/v1/auth/reset-password`	Reset Token	Body: `{reset_token, new_password}`. Verifies `purpose=="password_reset"` & checks Redis `used_reset_token:{jti}`, hashes new password, marks `jti` used.
POST	`/api/v1/auth/change-password`	Bearer JWT	Body: `{current_password|old_password, new_password}`. Verifies current password and updates Bcrypt hash for authenticated user.
GET PUT	`/api/v1/auth/preferences`	Bearer JWT	Gets or updates user UI preferences (`theme`, `language`, `notifications_enabled`).
GET	`/api/v1/auth/me`	Bearer JWT	Validates active status in DB and returns full user profile, schools, and role permissions array for `AuthBootstrap.jsx`.
POST	`/api/v1/auth/student/request-otp`	Public	Body: `{country_code, phone}` (E.164 7–15 digits). Matches active `student_profiles.guardian_phone`, issues `otp:student_login:{cc}{phone}`, logs OTP & emails guardian if real email exists.
POST	`/api/v1/auth/student/verify-otp`	Public	Body: `{country_code, phone, otp}`. Verifies student login OTP and returns 15-min `guardian_token` + serialized list of associated `students`.
POST	`/api/v1/auth/student/select`	Guardian Token	Body: `{guardian_token, student_id}`. Verifies student belongs to guardian phone and issues student's 24h `access_token` + student profile.

6.2 Users, Teachers, Classroom Operators & Students Router (`app/api/v1/users.py`)

Method(s)	Exact Route Path(s)	Capabilities & Query Parameters
GET POST PATCH DELETE	`/api/v1/user/school` `/api/v1/user/school/{user_id}`	Full CRUD for **School Admins**. Supports `?page=` , `?limit=` , `?search=` , `?status=` , `?school_id=` , and `?unassigned=true` (filters out School Admins already mapped to an active school).
GET POST PATCH DELETE	`/api/v1/user/content` `/api/v1/user/content/{user_id}`	Full CRUD for **Content Admins** (`content_admin` / `content_manager`).
GET POST PATCH DELETE	`/api/v1/users` `/api/v1/users/{user_id}`	Generic multi-role User directory & CRUD (supports `?role=` , `?school_id=` , `?status=` , `?search=`).
GET POST PATCH DELETE	`/api/v1/user/teacher` `/api/v1/user/teacher/{teacher_id}` `/api/v1/teacher/class` & `/teacher/subject`	Full CRUD for **Teachers**, managing qualification mappings (`teacher_qualifications`) and school-class-subject assignments (`teacher_assignments`).
GET POST	`/api/v1/teachers/bulk-upload/template` `/api/v1/teachers/bulk-upload/preview` `/api/v1/teachers/bulk-upload/commit`	3-stage CSV Bulk Upload pipeline for Teachers: downloads CSV template, parses & validates rows in preview mode, and commits valid rows transactionally.
GET POST PATCH DELETE	`/api/v1/user/class-room` `/api/v1/user/class-room/{operator_id}` `/api/v1/classroom-operator/class` & `/subject`	Full CRUD for **Classroom Operators**, managing school-class-section assignments (`classroom_operator_assignments`).
GET POST	`/api/v1/classroom-operators/bulk-upload/template` `/api/v1/classroom-operators/bulk-upload/preview` `/api/v1/classroom-operators/bulk-upload/commit`	3-stage CSV Bulk Upload pipeline for Classroom Operators.
GET POST PATCH DELETE	`/api/v1/user/student` `/api/v1/user/student/{student_id}`	Full CRUD for **Students**, managing `student_profiles` (roll number, DOB, academic year, class, section, and international E.164 guardian phone/email).
GET POST	`/api/v1/students/bulk-upload/template` `/api/v1/students/bulk-upload/preview` `/api/v1/students/bulk-upload/commit`	3-stage CSV Bulk Upload pipeline for Students.

6.3 Schools, Locations, Academics & Dashboard Routers

Router File	Mounted Route Paths (Dual Aliases)	Key Features & Business Logic
`app/api/v1/schools.py`	`GET/POST /api/v1/school` (and `/schools`) `GET/PATCH/DELETE /api/v1/school/{id}` `GET /api/v1/school/export` (CSV export) `GET/PATCH /api/v1/school/{id}/class-section`	Protected by `get_current_admin_user_id` for mutations. Enriches every school with live non-deleted `teacher_count` , `student_count` , `state_name` , `district_name` , `ulb_name` , `ward_name` , `region` , and assigned School Admins.
`app/api/v1/locations.py`	`GET/POST/PATCH/DELETE /api/v1/state` & `/academics/states` `GET/POST/PATCH/DELETE /api/v1/district` & `/academics/districts` `GET/POST/PATCH/DELETE /api/v1/ulb` & `/academics/ulbs` `GET/POST/PATCH/DELETE /api/v1/ward` & `/academics/wards`	Hierarchical CRUD for States → Districts (`?state_id=`) → ULBs (`?district_id=`) → Wards (`?ulb_id=`) with pagination and search.

Router File	Mounted Route Paths (Dual Aliases)	Key Features & Business Logic

`app/api/v1/academics.py`	`/api/v1/class` & `/academics/classes`
	`/api/v1/section` & `/academics/sections`
	`/api/v1/academics/streams`
	`/api/v1/subject` & `/academics/subjects`
	`/api/v1/academics/chapters` & `/academics/topics`
	`/api/v1/qualification` & `/academics/qualifications`
		Complete curriculum hierarchy CRUD (Classes, Sections, Streams, Subjects, Subject-Sections, Chapters, Topics, Teacher Qualifications).

`app/api/v1/dashboard.py`	`GET /api/v1/dashboard/summary` & `/reports/summary`
	`GET /api/v1/role-permissions/matrix`
	`PATCH/PUT /api/v1/role-permissions/{role_id}`
		Aggregates system-wide KPIs (schools, teachers, students, operators, content admins, state distribution) and serves/updates the 6-role RBAC permission matrix.

7. Super Admin Security Audit & Precautions (Items #1–#14)

Following Phase 2, a 14-point security and data-integrity audit was executed across all backend services and frontend components:

Audit #	Target Files	Vulnerability / Bug Found	Engineering Fix Implemented
#1–#5	`sql/init.sql`, `otp_service.py`, `auth_service.py`	Phase 1 to Phase 2 schema upgrades, 2FA resend-otp session bypass, bcrypt 72-byte crash, SMTP cooldown lock.	Idempotent 21-table DDL, Redis `otp_login_session:{email}` check, 72-byte UTF-8 validator, `revoke_otp()` + background SMTP tasks.
#6	`app/api/v1/auth.py`, `app/api/v1/schools.py`, `dashboard.py`	`get_current_user_id` previously trusted the JWT signature without verifying whether the user had been soft-deleted or deactivated in PostgreSQL; non-admin roles could call school/dashboard admin routes.	Replaced with `get_current_user` (queries DB and enforces `not user.is_deleted` and `user.status == 'ACTIVE'` on every request) and `get_current_admin_user_id` / `require_roles()` returning `403 Forbidden` for unauthorized roles.
#7	`app/services/school_service.py`	`create_school` and `update_school` accepted arbitrary UUIDs in `school_admin_ids` without checking existence or role.	Added `_validate_school_admin_ids(db, admin_ids)`: raises `404 Not Found` if any user UUID is missing/deleted, and `400 Bad Request` if `user.user_type ≠ "school_admin"`.
#8	`app/schemas/users.py`	Creating or updating any user role with a >72-byte password triggered an unhandled 500 error in `bcrypt.hashpw()`.	Created `PasswordCheckMixin` inherited by all user creation/update schemas enforcing 8–72 UTF-8 bytes with a clean `422` error.
#9	`app/core/response.py`	Database foreign-key or unique constraint races raised raw SQLAlchemy `IntegrityError` (HTTP 500).	Added `@app.exception_handler(IntegrityError)` in `register_exception_handlers()` returning a clean `400 Bad Request` envelope.
#10	`app/services/school_service.py`	School listings either showed `0` for teacher/student counts or included soft-deleted users in the count.	Updated `_enrich_schools()` to join `User` with `User.is_deleted.is_(False)` when counting distinct teachers and students per school.
#11	`school_service.py`, `admin_user_service.py`, `teacher_service.py`, `student_service.py`	Soft-deleting a School or User left active rows in `user_schools`, `teacher_assignments`, `classroom_operator_assignments`, and `student_profiles`.	Implemented transactional cascading soft-deletes across all child mapping and profile tables whenever a school or user is deleted.
#12	`app/api/v1/users.py`, `admin_user_service.py`	The "Assign School Admin" dropdown when creating a school listed admins already assigned to other schools.	Added `?unassigned=true` query filter on `GET /api/v1/user/school` excluding users with active rows in `offlineai.user_schools`.
#13	`app/services/school_service.py`	School cards and detail modals lacked human-readable location names (only UUIDs were returned).	Enriched school responses with `state_name`, `district_name`, `ulb_name`, `ward_name`, and `region` (`"District, State"`).
#14	`offline_frontend/src/ ...`	Frontend/backend field alignment across Super Admin and School Admin portals.	Verified all table columns, filters, bulk upload previews, and modals end-to-end.

8. Global International Phone Numbers (ITU-T E.164) & Student Login

8.1 Why International Numbers (e.g. UAE `+971 507652362`) Previously Failed

Both the frontend (`src/utils/validation.js`) and backend (`app/schemas/users.py`, `app/schemas/school.py`) originally hardcoded Indian 10-digit validation (`PHONE_REGEX = /^\d{10}$/`). When entering a valid 9-digit UAE number (`507652362` with country code `+971`), validation rejected it. Furthermore, when splitting stored E.164 strings like `+971507652362` via `digits.slice(-10)`, the code sliced the last 10 digits (`1507652362`), stealing the `1` from `+971` and corrupting both the country code and subscriber number.

8.2 Global E.164 Solution Across Frontend & Backend

- **ITU-T E.164 Subscriber Number Length (7 to 15 Digits):**
  - Updated `PHONE_REGEX = /^\d{7,15}$/` and `sanitizePhoneDigits()` (up to 15 digits) in `D:\Work\TechnoExponent\offline_frontend\src\utils\validation.js`.
  - Updated `_clean_optional_phone()` in `app/schemas/users.py`, `app/schemas/school.py`, and `StudentRequestOtpRequest` / `StudentVerifyOtpRequest` in `app/schemas/auth.py` to accept **7 to 15 digits**.
- **60+ Global Country Dialing Codes & Longest-Prefix-First Splitter:**
  - Expanded `COUNTRY_CODES` in `src/components/common/PhoneNumberFields.jsx` and `src/utils/validation.js` to 60+ countries (UAE `+971`, India `+91`, US/Canada `+1`, UK `+44`, Saudi Arabia `+966`, Qatar `+974`, Oman `+968`, Kuwait `+965`, Bahrain `+973`, Singapore `+65`, Australia `+61`, Germany `+49`, etc.).
  - Rewrote `splitPhoneForForm()` in `validation.js` and `_normalize_phone_fields()` in `app/services/admin_user_service.py` to match known calling codes sorted **longest-prefix-first** (4-digit → 3-digit → 2-digit → 1-digit codes). `+971507652362` now deterministically splits into `country_code = "+971"` and `phone_number = "507652362"`.
- **Student Login Page International Guardian Phone Support:**
  - Updated `D:\Work\TechnoExponent\offline_frontend\src\features\auth\pages\StudentLoginPage.jsx` to render `<PhoneNumberFields />` with the country code selector and 7–15 digit phone input, passing `{country_code, phone}` to `POST /api/v1/auth/student/request-otp` and `POST /api/v1/auth/student/verify-otp`.
  - Implemented `_find_students_by_guardian_phone(db, country_code, phone)` in `app/services/auth_service.py` to match both split fields (`guardian_country_code` + `guardian_phone`) and combined E.164 strings (`+971507652362`).

9. Docker Containerization & Local-to-Docker DB Migration

9.1 Multi-Container Architecture (`docker-compose.yml`)

Container Name	Image / Dockerfile	Host → Container Port	Configuration & Networking Details
`offline_ai_postgres`	`postgres:16-alpine`	`5433` → `5432`	Mounts `./sql/init.sql` to `/docker-entrypoint-initdb.d/01-init.sql` and persists data in volume `postgres_data`. Host port `5433` avoids conflicting with local Windows Postgres on `5432`.
`offline_ai_redis`	`redis:7-alpine`	`6380` → `6379`	Runs with `--appendonly yes` persistence in volume `redis_data`. Host port `6380` avoids local port `6379` conflicts.
`offline_ai_backend`	`./offline_ai_backend/Dockerfile` (`python:3.11-slim`)	`8000` → `8000`	Loads `.env` via `env_file` and overrides `POSTGRES_SERVER=postgres`, `POSTGRES_PORT=5432`, `REDIS_HOST=redis`, `REDIS_PORT=6379`. Mounts `./app:/app/app` and `./sql:/app/sql` for live reload.
`offline_ai_frontend`	`./offline_frontend/Dockerfile` (Node 20 → `nginx:alpine`)	`3000` → `80`	Multi-stage build compiling Vite production bundle (`VITE_API_BASE_URL=http://localhost:8000`) served by Nginx with SPA `try_files $uri $uri/ /index.html;` routing.

9.2 How the Docker Startup `socket.gaierror` Issue Was Resolved

Root Cause: During initial container startup, `asyncpg.connect()` inside `ensure_db_schema()` (`app/core/database.py`) raised `socket.gaierror: [Errno -3] Temporary failure in name resolution`. Ensure-schema was attempting to connect before Postgres DNS/readiness settled or when host `.env` values conflicted.

Fix Implemented in `app/core/database.py` & `docker-compose.yml` :

- Added a 10-attempt connection retry loop (`for attempt in range(1, 11): ... await asyncio.sleep(1.5)`) inside `ensure_db_schema()` in `app/core/database.py` using explicit connection parameters (`host=settings.POSTGRES_SERVER`, `port=settings.POSTGRES_PORT`, `user=settings.POSTGRES_USER`, `password=settings.POSTGRES_PASSWORD`, `database=settings.POSTGRES_DB`).
- Configured `docker-compose.yml` with `depends_on: postgres: condition: service_healthy` and `redis: condition: service_healthy` so `offline_ai_backend` starts cleanly every time.

9.3 Lossless Migration from Local PostgreSQL 18 (`5432`) to Docker PostgreSQL 16 (`5433`)

1. Exported the complete `offlineai` schema and data from the host's local PostgreSQL 18 instance using `pg_dump -h localhost -p 5432 -U postgres -d auth_backend -n offlineai --no-owner --no-privileges`.
2. Stripped the PostgreSQL 18-only `SET transaction_timeout = 0;` directive for PostgreSQL 16 compatibility.
3. Imported the dump into `offline_ai_postgres` (`docker exec -i offline_ai_postgres psql -U postgres -d auth_backend`) and verified all 21 tables and records (10 Users, 2 Schools including `Global International School` with UAE `+971` phone, 5 States, 7 Districts, 4 ULBs, 5 Wards, 5 Classes, 10 Sections, 10 Subjects, 8 Qualifications, and 6 Role Permission sets).

10. Complete File Inventory & Operational Runbook

File Path	Architectural Role
`offline_ai_backend/AUTH_BUILD_SPEC.md`	Initial architecture specification defining the 5-layer service separation, frontend response envelope, Redis OTP rules, and Gmail SMTP architecture.
`offline_ai_backend/app/core/config.py`	Pydantic `Settings` loading Postgres, Redis, JWT, OTP (`OTP_LENGTH=6`, `OTP_EXPIRE_SECONDS=300`, `OTP_RESEND_COOLDOWN_SECONDS=30`, `OTP_MAX_ATTEMPTS=5`), and Gmail SMTP (`SMTP_HOST`, `SMTP_PORT=587`, `SMTP_DEV_MODE`).

File Path	Architectural Role
`offline_ai_backend/app/core/security.py`	Direct `bcrypt` hashing (12 rounds, 72-byte limit), `DUMMY_PASSWORD_HASH` anti-timing constant, and `TokenPurpose` JWT encoder/decoder (`ACCESS`, `PASSWORD_RESET`, `GUARDIAN_SELECT`).
`offline_ai_backend/app/core/response.py`	Standardized `{success, message, data, verification_required, errors}` envelope, `paginated_data()`, `AppError`, and global exception handlers.
`offline_ai_backend/app/services/otp_service.py`	Redis OTP generation (`secrets.choice`), `OtpPurpose` isolation, brute-force counter (`otp_attempts`), 30s cooldown (`otp_cooldown`), `otp_login_session`, and `revoke_otp()`.
`offline_ai_backend/app/services/mailer_service.py`	Generic `send_email(to, subject, body)` over Gmail STARTTLS (`aiosmtplib` port 587) with `SMTP_DEV_MODE` console printout fallback.
`offline_ai_backend/app/services/auth_service.py`	Orchestrates 2FA login, OTP verify/resend, forgot/reset password (with `used_reset_token:{jti}`), and international E.164 Student Guardian login.
`offline_ai_backend/app/services/*.py`	Domain services: `user_service.py`, `admin_user_service.py`, `school_service.py`, `teacher_service.py`, `student_service.py`, `classroom_operator_service.py`, `academic_service.py`, `location_service.py`, `dashboard_service.py`.
`offline_ai_backend/sql/init.sql`	Idempotent 21-table PostgreSQL DDL (`offlineai` schema) and default seed data.
`offline_ai_backend/Dockerfile` & `docker-compose.yml`	Container definitions and 4-service orchestration (`postgres`, `redis`, `backend`, `frontend`).

```
# Start the entire stack in Docker (from D:\Work\TechnoExponent\offline_ai_backend)
docker compose up --build -d

# View live backend logs (including [DEV MODE] OTP printouts & [STUDENT OTP] codes)
docker compose logs -f backend

# Stop all containers (preserving database & Redis volumes)
docker compose down
```

Default Credentials & Live Endpoints

- **Frontend Application:** `http://localhost:3000` | **Backend API & Swagger Docs:** `http://localhost:8000/docs`
- **Seeded Super Admin:** `humza.ahmad050504@gmail.com` | Password: `Test@1234`
- **Docker Postgres (Host Port):** `localhost:5433` (DB: `auth_backend`, Schema: `offlineai`) | **Docker Redis:** `localhost:6380`